Engenharia de Sistemas Distribuídos

TICKETFLOW: O DESAFIO DA VENDA DE INGRESSOS

Plataforma de venda de ingressos com reserva de assentos focada em alta concorrência e integridade de dados.

João Victor, Luerson, Leandeson, Herick, João Lucas e Luana.

O DESAFIO DA ALTA CONCORRÊNCIA

- Milhares de usuários disputando os mesmos assentos simultaneamente.
- Risco de Double Booking: Vender o mesmo assento duas vezes.
- Falhas de Rede: Cobrança em dobro por cliques repetidos.
- Inconsistência: Queda de serviços no meio do fluxo de compra.



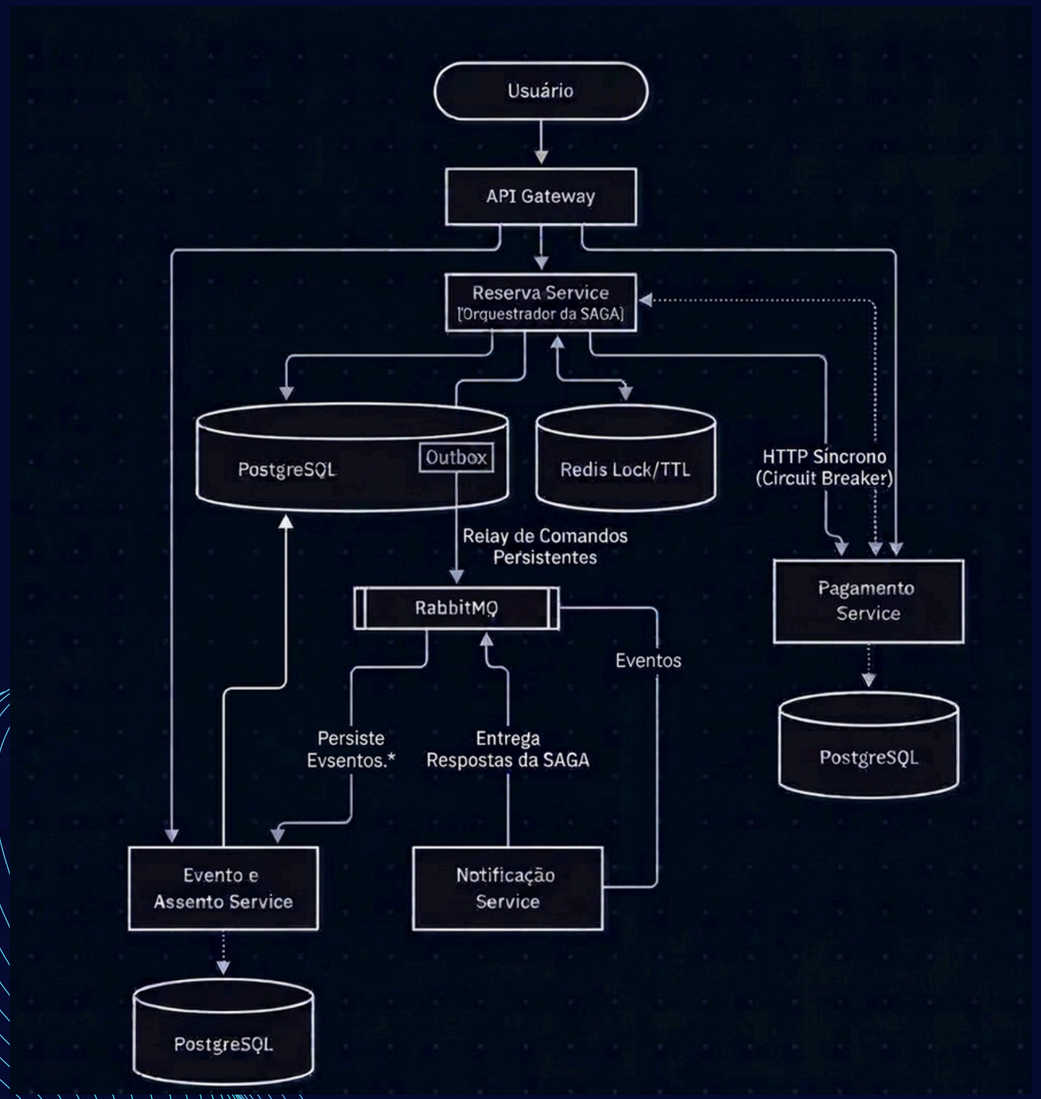
DECISÕES ARQUITETURAIS (ADRS)





- ADR-001 (RabbitMQ): Mensageria ponto a ponto, simples e com Dead Letter Queue.
- ADR-002 (SAGA Orquestrada): Fluxo centralizado no Reservas; pagamento via HTTP síncrono.
- ADR-003 (PostgreSQL é a Verdade): Redis é só lock temporário. O banco impede vendas duplicadas via escrita condicional (UPDATE...WHERE).
- Resiliência: Circuit Breaker (Resilience4j) isola falhas no Pagamento.





ARQUITETURA E PADRÕES

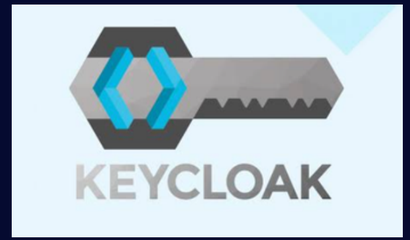
- API Gateway: Ponto único de entrada.
- SAGA: Orquestração no reserva-service (máquina de estados).
- Transactional Outbox: Garantia de entrega (RabbitMQ).
- Idempotência: Proteção contra cobranças duplicadas.

DEMAIS STACKS





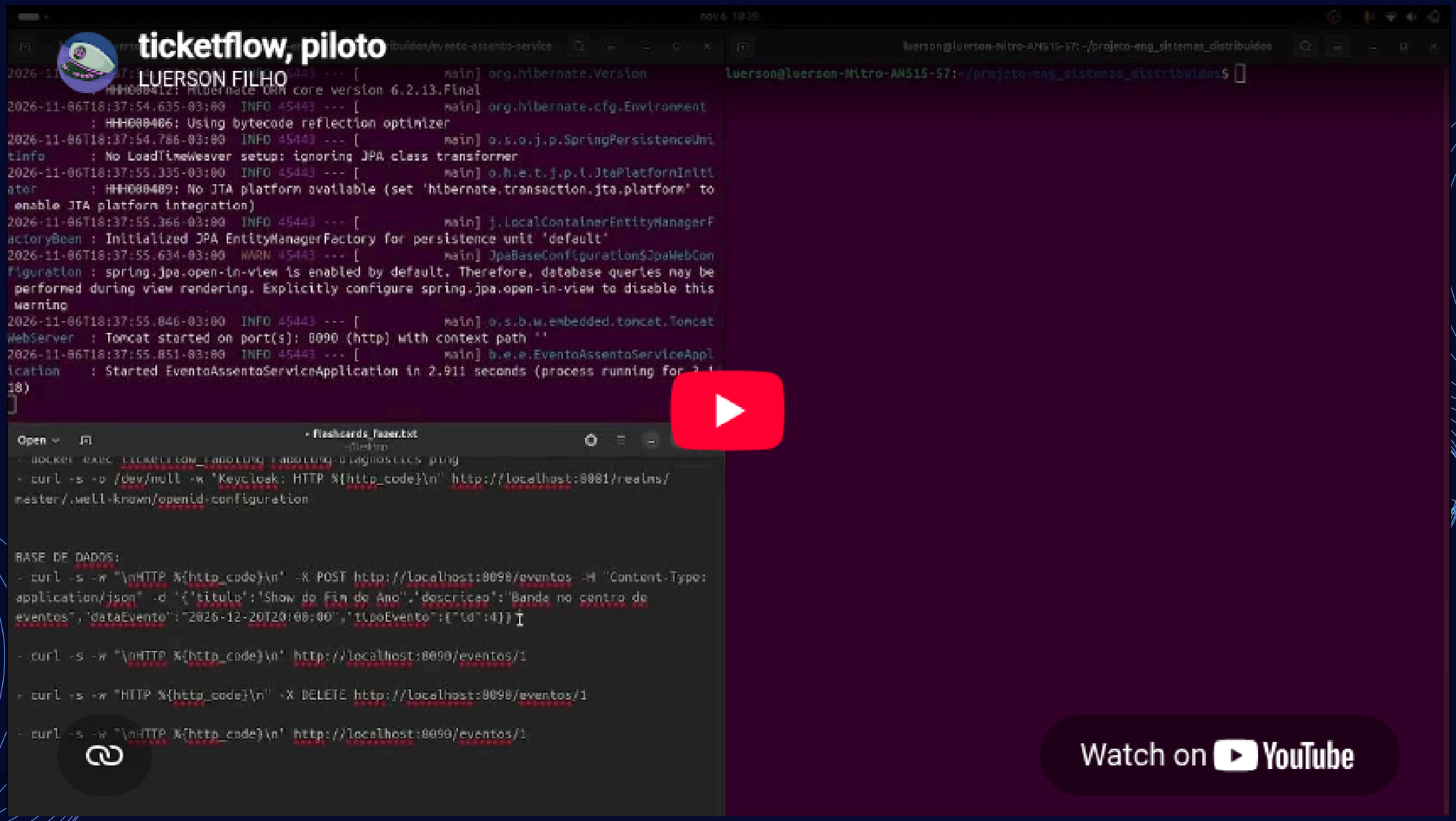
- SpringBoot: Facilita o desenvolvimento de microsserviços com configuração automática, servidor embutido e ecossistema maduro para APIs REST.
- Spring Cloud Gateway: Implementa o padrão API Gateway, atuando como ponto único de entrada para roteamento, autenticação, autorização e rate limiting.
- Keycloack: Solução de gerenciamento de identidade utilizada para autenticação baseada em OAuth 2.0 e JWT, garantindo a segurança do sistema.



STACK TECNOLÓGICA

	Tecnologia	Por que foi escolhida?
	**Spring Boot**	Facilita o desenvolvimento de microsserviços com configuração automática, servidor embutido e ecossistema maduro para APIs REST.
	**Spring Cloud Gateway**	Implementa o padrão API Gateway, atuando como ponto único de entrada para roteamento, autenticação, autorização e rate limiting.
	**Resilience4j**	Biblioteca utilizada para implementar o padrão Circuit Breaker, permitindo isolar falhas e evitar a propagação de erros em cascata.
	**PostgreSQL**	Banco relacional escolhido para garantir propriedades ACID nas transações de cada serviço, fundamental para a consistência e para o padrão Transactional Outbox.
	**Redis**	Banco em memória escolhido para alta performance, sendo utilizado para gerenciar a reserva temporária dos assentos (com TTL) e o rate limiting.
	**RabbitMQ**	Broker de mensagens utilizado para a comunicação assíncrona entre os serviços, viabilizando o Transactional Outbox.
	**Keycloak**	Solução de gerenciamento de identidade utilizada para autenticação baseada em OAuth 2.0 e JWT, garantindo a segurança do sistema.
	**Docker Compose**	Ferramenta para orquestração do ambiente local, permitindo subir toda a infraestrutura e os microsserviços de forma padronizada e isolada.
	**GitHub Actions**	Plataforma de CI/CD escolhida para automatizar a integração contínua, rodando testes e garantindo a qualidade do código a cada Pull Request.
	**Flyway**	Ferramenta escolhida para gerenciar as migrações e o versionamento dos schemas de banco de dados de cada microserviço.
	**MapStruct**	Utilizado para o mapeamento dos objetos (ex: DTOs para Entidades) de forma performática e segura em tempo de compilação.

DEMONSTRAÇÃO



GITHUB

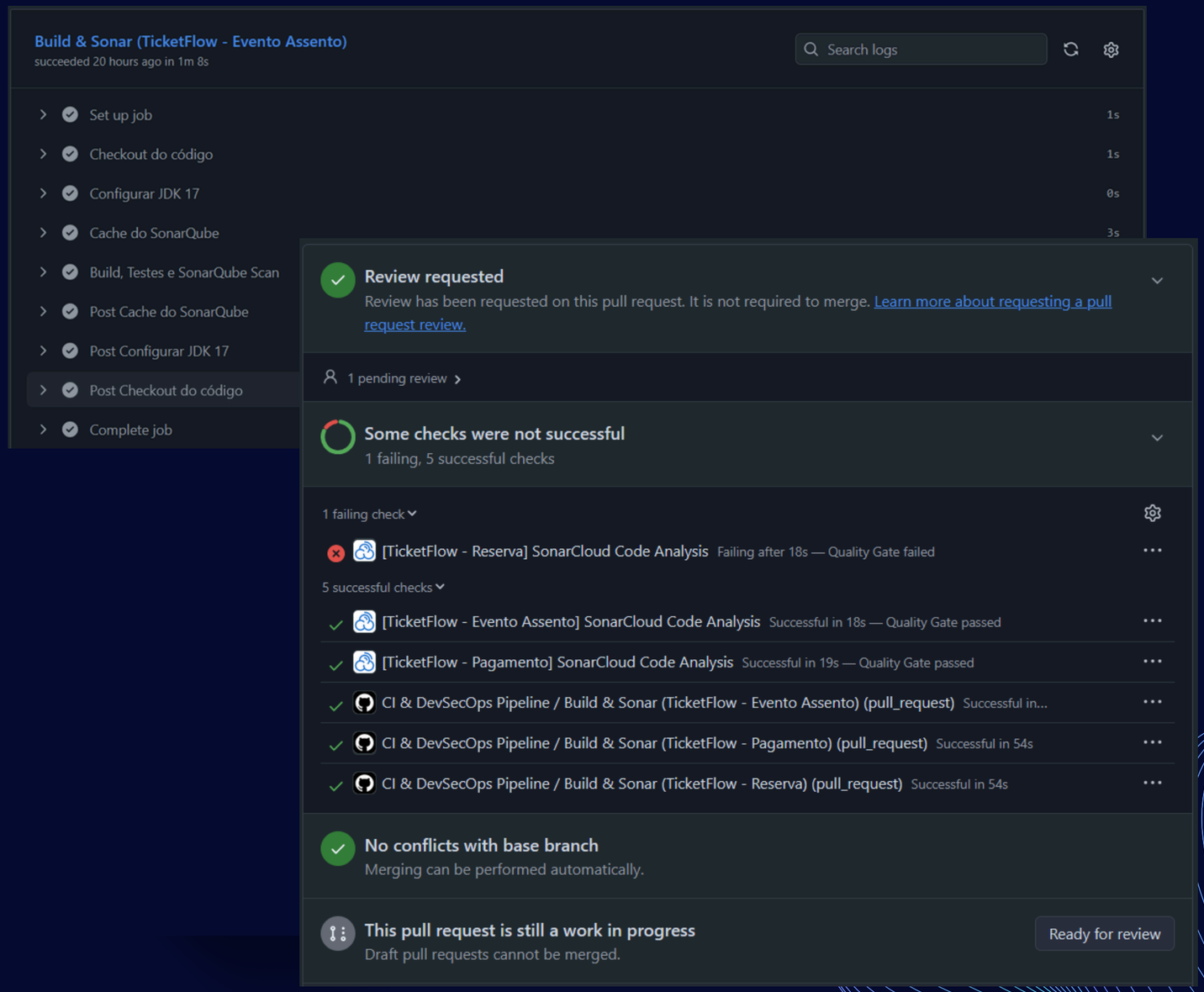
- GitHub Actions para a criação do pipeline CI;

Job 01 (Build & Sonar):

1. Faz o checkout do código para o runner do GitHub;
2. Build do sistema no runner + configurações;
3. Verificação de qualidade estática do código fonte com o SonarQube (SAST);

Job 02 (Qualite Gate - SonarQube):

1. Análise da cobertura de testes de integração e unidade de novos códigos em pull requests e pushes.



LIÇÕES APRENDIDAS

Reflexões da equipe sobre os desafios de transpor a teoria de sistemas distribuídos para a prática.

01

Complexidade de Transações Distribuídas

Modelar padrões como SAGA, Outbox e Idempotência nos ADRs mostrou o tamanho do desafio de garantir a consistência dos dados sem uma transação única de banco.

02

O Poder da Documentação:

A criação dos ADRs e dos diagramas C4 foi essencial para nivelar o conhecimento dos 6 integrantes e planejar a arquitetura antes de partir para o código.

03

Desafios da Infraestrutura

Configurar o ecossistema de microsserviços e a orquestração local com o Docker Compose provou a complexidade de gerenciar múltiplos componentes (Bancos, Redis, Broker).